FIN2020 HW2 文杰 123090612

1. Expected Utility and Risk Aversion

The wealth outcomes are:

- **Lottery 1 (risky):** 15 w.p. $\frac{1}{2}$, 10 w.p. $\frac{1}{2}$
- **Lottery 2 (safe):** 12.5 for sure
- **Lottery 3 (safe):** 12 for sure

1) Compute U for $\gamma = \frac{1}{2}$

When $\gamma = \frac{1}{2}$, $1-\gamma = \frac{1}{2}$, so

$$u(c) = \frac{c^{\frac{1}{2}}}{\frac{1}{2}} = 2\sqrt{c}.$$

- **Lottery 1**

$$U_1 = \tfrac{1}{2}\cdot 2\sqrt{15} + \tfrac{1}{2}\cdot 2\sqrt{10} \approx 7.0353.$$

- **Lottery 2**

$$U_2 = 2\sqrt{12.5} \approx 7.0711.$$

- **Lottery 3**

$$U_3 = 2\sqrt{12} \approx 6.9282.$$

Ranking ($\gamma = \frac{1}{2}$): $\boxed{U_2 > U_1 > U_3}$.

2) Compute U for $\gamma = 2$

When $\gamma = 2$, $1-\gamma = -1$, so

$$u(c) = \frac{c^{-1}}{-1} = -\frac{1}{c}.$$

- **Lottery 1**

$$U_1 = \tfrac{1}{2}\Big(-\frac{1}{15}\Big) + \tfrac{1}{2}\Big(-\frac{1}{10}\Big) = -\frac{1}{12} \approx -0.08333.$$

- **Lottery 2**

$$U_2 = -\frac{1}{12.5} = -0.08.$$

- **Lottery 3**

$$U_3 = -\frac{1}{12} \approx -0.08333.$$

Ranking ($\gamma = 2$): $\boxed{U_2 > U_1 = U_3}$.

3) Compute U for $\gamma = 3$

When $\gamma = 3$, $1-\gamma = -2$, so

$$u(c) = \frac{c^{-2}}{-2} = -\frac{1}{2c^2}.$$

- **Lottery 1**

$$U_1 = \tfrac{1}{2}\Big(-\frac{1}{2\cdot 15^2}\Big) + \tfrac{1}{2}\Big(-\frac{1}{2\cdot 10^2}\Big) = -\frac{13}{3600} \approx -0.0036111.$$

- **Lottery 2**

$$U_2 = -\frac{1}{2\cdot 12.5^2} = -\frac{1}{312.5} \approx -0.0032.$$

- **Lottery 3**

$$U_3 = -\frac{1}{2\cdot 12^2} = -\frac{1}{288} \approx -0.0034722.$$

Ranking ($\gamma = 3$): $\boxed{U_2 > U_3 > U_1}$.

4) Does the investor ever prefer $(5, 0, \frac{1}{2})$ to $(2.5, 0, 1)$?

No (for all $\gamma > 0$). For CRRA with $\gamma > 0$, $u''(c) = -\gamma c^{-\gamma-1} < 0$, so u is concave (risk aversion). Then by **Jensen's inequality** for concave u,

$$u\big(W_0 + E[\tilde{z}]\big) \;\geq\; E\big[u(W_0 + \tilde{z})\big],$$

with strict inequality for a non-degenerate gamble. That is: the sure payoff equal to the mean is (weakly/strictly) preferred to the risky payoff with the same mean.

Here, Lottery 1 has $E[\tilde{z}] = 2.5$, which is exactly Lottery 2's sure payoff, so a risk-averse investor prefers Lottery 2, consistent with the computed values $U_2 > U_1$ for $\gamma = \frac{1}{2}, 2, 3$.

5) Compare $(5, 0, \frac{1}{2})$ to $(2, 0, 1)$

From the computed utilities:

- $\gamma = \frac{1}{2}$**:** $U_1 \approx 7.0353 > U_3 \approx 6.9282 \Rightarrow$ **prefers risky**.
- $\gamma = 2$**:** $U_1 = U_3 = -1/12 \Rightarrow$ **indifferent**.
- $\gamma = 3$**:** $U_1 \approx -0.0036111 < U_3 \approx -0.0034722 \Rightarrow$ **prefers safe ($2)**.

So:

$$\boxed{\text{Prefers risky at } \gamma = \tfrac{1}{2}; \quad \text{indifferent at } \gamma = 2; \quad \text{prefers safe at } \gamma = 3.}$$

Why this pattern? In CRRA utility, γ is the **coefficient of (constant) relative risk aversion**, so larger γ means more curvature and a larger "penalty" for dispersion in outcomes.
A helpful summary is the **certainty equivalent** (CE) of Lottery 1, defined by

$$u(W_0 + CE) = E[u(W_0 + \tilde{z})].$$

For Lottery 1:

- $CE(\gamma = \frac{1}{2}) \approx 2.374$ (so $CE > 2$: choose risky over $2 sure),
- $CE(\gamma = 2) = 2.000$ (so $CE = 2$: indifference),
- $CE(\gamma = 3) \approx 1.767$ (so $CE < 2$: choose $2 sure).

---

2. Interpreting Measures of Risk Aversion

Under vN–M expected utility, indifference between **rejecting** the bet (keeping certain wealth Y_0) and **accepting** the bet (wealth $Y_0 \pm h$) occurs when

$$u(Y_0) = \pi^* u(Y_0 + h) + (1 - \pi^*) u(Y_0 - h),$$

so solving for π^* yields the **exact** indifference probability

$$\boxed{\pi^* = \frac{u(Y_0) - u(Y_0 - h)}{u(Y_0 + h) - u(Y_0 - h)}}$$

(using linearity of expected utility in probabilities).

Here $Y_0 = 10$ and the lottery $(0.1, -0.1, \pi)$ implies $h = 0.1$ (a $\pm$\$0.1 bet). The bet size as a fraction of wealth is

$$k = \frac{h}{Y_0} = \frac{0.1}{10} = 0.01.$$

The Bernoulli utility is CRRA:

$$u(Y) = \frac{Y^{1-\gamma} - 1}{1 - \gamma}, \qquad \gamma > 0.$$

For CRRA, the coefficient of relative risk aversion is constant and equals γ, i.e.

$$-\frac{Y u''(Y)}{u'(Y)} = \gamma,$$

so the class approximation becomes

$$\boxed{\pi^* \approx \frac{1}{2} + \frac{1}{4}\gamma k = \frac{1}{2} + 0.0025\gamma}$$

(with $k = 0.01$).

Numerical results ($Y_0 = 10,\ h = 0.1,\ k = 0.01$)

PYTHON

```python
import math
from typing import List, Tuple

def exact_pi_star(y0: float, h: float, gamma: float) -> float:
    """
    Exact indifference probability for lottery (h, -h, pi) at wealth y0
    under CRRA utility.

    Uses a numerically stable reformulation to avoid subtracting 1 inside CRRA utility.

    For gamma != 1:
        u(y) = (y^(1-gamma) - 1)/(1-gamma) = (1 - y^(1-gamma))/(gamma-1)

    Plugging into:
        pi* = (u(y0) - u(y0-h)) / (u(y0+h) - u(y0-h))

    The constant (gamma-1) cancels, yielding:
        pi* = ((y0-h)^(1-g) - y0^(1-g)) / ((y0-h)^(1-g) - (y0+h)^(1-g))

    For gamma == 1 (log utility):
        u(y)=ln y, so:
        pi* = (ln y0 - ln(y0-h)) / (ln(y0+h) - ln(y0-h))
    """
    if y0 <= 0:
        raise ValueError("y0 must be positive.")
    if h <= 0:
        raise ValueError("h must be positive.")
    if y0 - h <= 0:
        raise ValueError("y0 - h must be positive (wealth must stay positive).")

    # Log utility special case
    if abs(gamma - 1.0) < 1e-12:
        u0 = math.log(y0)
        u_minus = math.log(y0 - h)
        u_plus = math.log(y0 + h)
        denom = u_plus - u_minus
        if denom == 0.0:
            raise ZeroDivisionError("Denominator is exactly zero; check parameters.")
        return (u0 - u_minus) / denom

    # Stable CRRA-based expression
    p = 1.0 - gamma  # exponent
    a = (y0 - h) ** p
    b = (y0) ** p
    c = (y0 + h) ** p

    denom = a - c
    if denom == 0.0:
        raise ZeroDivisionError("Denominator is exactly zero; check parameters.")
    return (a - b) / denom

def approx_pi_star(y0: float, h: float, gamma: float) -> float:
    """
    Approximation from class:
        pi* ≈ 1/2 + 1/4 * RRA(y0) * k
    For CRRA, RRA(y)=gamma and k=h/y0, so:
        pi* ≈ 1/2 + 1/4 * gamma * (h/y0)
    """
    k = h / y0
    return 0.5 + 0.25 * gamma * k

def compute_table(
    gammas: List[float],
    y0: float = 10.0,
    h: float = 0.1
) -> List[Tuple[float, float, float, float]]:
    """
    Returns rows of:
        (gamma, exact_pi, approx_pi, exact_minus_approx)
    """
    rows = []
    for g in gammas:
        pi_exact = exact_pi_star(y0, h, g)
        pi_approx = approx_pi_star(y0, h, g)
        rows.append((g, pi_exact, pi_approx, pi_exact - pi_approx))
    return rows

def format_rows_markdown(rows: List[Tuple[float, float, float, float]]) -> str:
    """
```

```
    Formats rows into a Markdown table.
    """
    lines = []
    lines.append("| gamma | exact pi* | approx pi* | exact - approx |")
    lines.append("|---:|---:|---:|---:|")
    for g, pe, pa, diff in rows:
        lines.append(f"| {g:g} | {pe:.10f} | {pa:.10f} | {diff:.10e} |")
    return "\n".join(lines)

if __name__ == "__main__":
    gammas = [0.5, 2, 3, 10, 20]
    rows = compute_table(gammas, y0=10.0, h=0.1)
    print(format_rows_markdown(rows))
```

Results:

gamma	exact pi*	approx pi*	exact - approx
0.5	0.5012500234	0.5012500000	2.3438577967e-08
2	0.5050000000	0.5050000000	-1.9984014443e-15
3	0.5074997500	0.5075000000	-2.4999999937e-07
10	0.5249816812	0.5250000000	-1.8318800054e-05
20	0.5498430691	0.5500000000	-1.5693085556e-04